

Planejamento Completo de Estudos — Linguagem Rust

PLANEJAMENTO COMPLETO DE ESTUDOS — LINGUAGEM RUST

INTRODUÇÃO

Rust é uma linguagem moderna focada em segurança de memória, alta performance, concorrência segura e confiabilidade. Ela vem ganhando espaço em sistemas embarcados, backend, firmware, IoT e aplicações de alta performance.

OBJETIVO DO PLANEJAMENTO

Este planejamento foi estruturado para:

- Desenvolver uma base sólida na linguagem Rust;
- Compreender profundamente ownership e borrowing;
- Construir projetos reais;
- Preparar o aluno para Embedded Rust;
- Integrar conhecimentos prévios de sistemas embarcados.

ESTRUTURA GERAL

FASE 1 — FUNDAMENTOS DO RUST

Duração: 2 semanas

Semana 1:

- Instalação do ambiente
- Cargo e estrutura de projetos
- Variáveis e tipos
- Controle de fluxo
- Ownership
- Borrowing

Semana 2:

- Structs
- Enums
- Option e Result
- Collections
- Modules
- Projeto CLI simples

FASE 2 — RUST INTERMEDIÁRIO

Duração: 2 semanas

Conteúdos:

- Lifetimes
- Traits
- Generics
- Threads
- Channels
- Smart pointers
- Async/Await
- Tokio

FASE 3 — PROJETOS PRÁTICOS

Duração: 2 semanas

Projeto 1:

CLI Profissional

- clap
- serde
- toml

Projeto 2:

Cliente MQTT

- rumqtte
- tokio

- TLS

- reconexão

Projeto 3:

Simulador embarcado

- máquina de estados
- timers
- concorrência

FASE 4 — EMBEDDED RUST

Bibliotecas importantes:

- embedded-hal
- embassy
- probe-rs
- defmt

Ordem recomendada:

1. GPIO

2. UART

3. I2C

4. SPI

5. PWM

6. DMA

7. WiFi

8. MQTT

FERRAMENTAS IMPORTANTES

Editor:

- Visual Studio Code

Extensões:

- rust-analyzer
- CodeLLDB
- Better TOML
- Error Lens

Comandos importantes:

cargo new projeto

cargo build

cargo run

cargo check

cargo fmt

cargo clippy

CONCEITOS FUNDAMENTAIS

OWNERSHIP

Ownership define:

- quem é dono dos dados;
- quando os dados serão destruídos;
- como evitar problemas de memória.

BORROWING

Permite acessar dados sem transferir ownership.

LIFETIMES

Define quanto tempo uma referência permanece válida.

TRAITS

Funcionam de forma semelhante a interfaces.

GENERICIS

Permitem reutilização de código.

CRATES IMPORTANTES

- tokio
- serde
- serde_json
- reqwest
- clap
- anyhow
- rumqttc

PROJETOS RECOMENDADOS

1. Sistema MQTT
2. Monitor serial
3. Logger de sensores
4. Firmware async
5. Máquina de estados

CRONOGRAMA RECOMENDADO

Semana 1:

Sintaxe e ownership

Semana 2:

Structs, enums e Result

Semana 3:

Traits, lifetimes e generics

Semana 4:

Async, threads e crates

Semana 5:

Projeto MQTT

Semana 6:

Projeto embarcado simulado

Semana 7+:

Embedded Rust

RECURSOS OFICIAIS

Livro oficial:

<https://doc.rust-lang.org/book/>

Rustlings:

<https://github.com/rust-lang/rustlings>

Embedded Rust:

<https://docs.rust-embedded.org/book/>

Exercism:

<https://exercism.org/tracks/rust>

CONCLUSÃO

Rust possui curva de aprendizado mais exigente no início, principalmente devido a ownership, borrowing e lifetimes.

Porém, após dominar esses conceitos, a produtividade cresce significativamente.

Para sistemas embarcados, Rust oferece:

- segurança;
- concorrência segura;
- confiabilidade;
- excelente desempenho.